

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	B.MANU JASWANTH	Reg. No.:	192525400
Section / Batch:	2025	Date:	28/07/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I B.MANU JASWANTH/192525400(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: MANU

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----	---------------------	----------------------	----------------------	---------------------

Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Debugging & Optimisation Task

Problem Identification

The given C program is intended to find two array elements whose sum equals the target value using the **Brute Force Pair Sum** approach.

Errors Identified

1. Incorrect Output Assignment

- The function stores the indices in reversed output variables.
- Current code:
 - `*b = i;`
 - `*a = j;`
- Expected:
 - `*a = i;`
 - `*b = j;`

2. Output Consistency Issue

- The caller expects a to contain the first index and b to contain the second index.
- Reversing them produces confusing and incorrect results.

3. Maintainability Problem

- Future developers may misunderstand which variable stores which position.
- Debugging becomes difficult because output does not match variable names.

Debugging Process

Step 1 – Examine the Logic

The algorithm checks every possible pair.

Outer Loop → First element

Inner Loop → Second element

When

`arr[i] + arr[j] == target`

a valid pair has been found.

Step 2 – Identify the Bug

Original code

`*b = i;`

`*a = j;`

Here,

- first index is stored in **b**
- second index is stored in **a**

This is opposite to the intended output parameters.

Step 3 – Correct the Assignment

Correct code

`*a = i;`

`*b = j;`

Now

- **a** → first position
- **b** → second position

Output becomes consistent.

Step 4 – Verify the Output

Example

Array = {2,7,11,15}

Target = 9

Comparison

$2 + 7 = 9$

Correct Output

a = 0

b = 1

Optimization

The algorithm is optimized by checking only **unordered pairs**.

Inner loop starts from

$j = i + 1$

instead of

$j = 0$

Therefore,

- Pair (0,1) is checked once.
- Pair (1,0) is never checked again.
- No element is compared with itself.

This removes unnecessary comparisons and improves efficiency while preserving the brute-force approach.

Time Complexity Analysis

Case	Time Complexity	Explanation
Best Case	$O(1)$	Pair found during the first comparison.
Average Case	$O(n^2)$	Approximately half of all pairs are checked.
Worst Case	$O(n^2)$	Every unordered pair is checked before returning.

Space Complexity

$O(1)$

Only a few variables are used.

Corrected C Program

```
#include <stdio.h>
```

```
/*
```

```
Function : pairSum
```

```
Purpose : Find two array elements whose sum equals the target.
```

```
Returns : 1 if pair is found, otherwise 0.
```

```
*/
```

```
int pairSum(int arr[], int n, int target, int *a, int *b)
```

```
{
```

```

int i, j;

// Check each unordered pair only once
for(i = 0; i < n - 1; i++)
{
    for(j = i + 1; j < n; j++)
    {
        // Pair found
        if(arr[i] + arr[j] == target)
        {
            // Store indices correctly
            *a = i;
            *b = j;

            return 1;
        }
    }
}

// No valid pair found
return 0;
}

int main()
{
    int arr[] = {2, 7, 11, 15};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 9;

    int firstIndex, secondIndex;

    if(pairSum(arr, n, target, &firstIndex, &secondIndex))
    {
        printf("Pair Found at Indices: %d and %d\n",
            firstIndex, secondIndex);

        printf("Values: %d and %d\n",
            arr[firstIndex], arr[secondIndex]);
    }
    else
    {
        printf("Pair Not Found\n");
    }

    return 0;
}

```

Sample Output

Pair Found at Indices: 0 and 1

Values: 2 and 7

Analysis and Justification

- The logical bug was identified by inspecting the output parameter assignments.
- The output variables were corrected to store the indices in the intended order (*a = i, *b = j).
- The implementation checks each unordered pair only once ($j = i + 1$), avoiding duplicate comparisons.
- The corrected algorithm maintains the brute-force approach while improving clarity and efficiency.
- The final code is clean, properly commented, and easy to debug, satisfying the assessment requirements for **Problem Identification, Debugging, Optimization, Analysis, and Code Quality**.